Reviewer Pack — Multiplicity Gap in Symmetric Powers of Permanent vs Determi…

Entry 2746a7cbee6d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 15:45:52 UTC

Multiplicity Gap in Symmetric Powers of Permanent vs Determinant Representations

Entry ID: `2746a7cbee6d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 15:45:52 UTC

1. Conjecture

Field A (mathematical branch): SCHUR_WYEL_DUALLY **Field B** (complexity object): PERMANENT_DETERMINANT_COMPLEXITY

Statement:

> For all integers m < n^{1.5}, the multiplicity of the irreducible representation indexed by the partition (n - m, m) in the symmetric power S^m(perm_n) exceeds its multiplicity in S^m(det_n), where perm_n and det_n denote the permanent and determinant polynomials of size n.

Rationale (proposer's reasoning):

> This conjecture leverages Schur-Weyl duality to identify a representation-theoretic invariant that distinguishes the symmetric structures of permanent and determinant polynomials. The multiplicity gap in symmetric powers could provide a geometric obstruction to embedding the permanent's orbit within the determinant's orbit closure, aligning with GCT's separation strategy.

Taxonomy category: `META_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9062ee40476a81db`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def factorial(n):
        if n == 0 or n == 1:
            return 1
        else:
            return n * factorial(n - 1)

    def binomial_coefficient(n, k):
        return factorial(n) // (factorial(k) * factorial(n - k))

    def young_tableau_to_permutation(yt):
        perm = [0] * len(yt)
        for i in range(len(yt)):
            for j in range(len(yt[i])):
                perm[yt[i][j]] = i * len(yt[i]) + j
        return perm

    def permanent(matrix):
        if len(matrix) == 1:
            return matrix[0][0]
        else:
            det = 0
            sign = 1
            for j in range(len(matrix)):
                submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
                det += sign * matrix[0][j] * permanent(submatrix)
                sign *= -1
            return det

    def determinant(matrix):
        if len(matrix) == 1:
            return matrix[0][0]
        else:
```

```
        det = 0
        for j in range(len(matrix)):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += ((-1) ** j) * matrix[0][j] * determinant(submatrix)
        return det

    def plethysm(m, n):
        if m == 0:
            return 1
        else:
            return sum(binomial_coefficient(n, k) * plethysm(m - 1, k) for k in range(1, n + 1))

    def multiplicity(partition, matrix):
        n = len(matrix)
        if partition[0] > n or any(x < 0 for x in partition):
            return 0
        yt = []
        for i in range(len(partition)):
            row = [i * (n // partition[i]) + j for j in range(partition[i])]
            yt.append(row)
        perm = young_tableau_to_permutation(yt)
        det = determinant(matrix)
        perm_val = permanent(matrix)
        return binomial_coefficient(n, len(perm)) * plethysm(len(partition), n) // (det ** len(par

    def partition_to_tuple(partition):
        return tuple(sorted(partition, reverse=True))

    def partitions(n):
        if n == 0:
            yield []
        else:
            for i in range(1, n + 1):
                for p in partitions(n - i):
                    yield [i] + p

    results = []
    n_max = 40
    for n in range(5, n_max + 1):
        m_max = int(n ** 1.5)
        for m in range(1, m_max):
            perm_multiplicity = multiplicity((n - m, m), [[random.randint(-10, 10) for _ in range(
            det_multiplicity = multiplicity((n - m, m), [[random.randint(-10, 10) for _ in range(n
            results.append({
                "metric_name": "Multiplicity Gap",
                "metric_value": perm_multiplicity - det_multiplicity,
                "instances_tested": 1,
                "conjecture_holds": perm_multiplicity > det_multiplicity,
                "counterexample": f"m={m}, n={n}: perm_multiplicity={perm_multiplicity}, det_multi
            })

    return {
        "metric_name": "Multiplicity Gap",
        "metric_value": sum(x["metric_value"] for x in results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": all(x["conjecture_holds"] for x in results),
```

```
        "counterexample": next((x["counterexample"] for x in results if not x["conjecture_holds"])
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = [run_trial(seed) for seed in seeds]

    print("TRIALS:")
    for result in results:
        print(f"TRIAL: {result}")

    mean_value = sum(x["metric_value"] for x in results) / len(results)
    std_value = (sum((x["metric_value"] - mean_value) ** 2 for x in results) / len(results)) ** 0.
    support_fraction = sum(1 for x in results if x["conjecture_holds"]) / len(results)

    if all(x["conjecture_holds"] for x in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not x["conjecture_holds"] for x in results):
        first_failing_seed = next(x['seed'] for x in results if not x['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{next(x['counterexample'] for x in results if n
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa1ea576.py", line 115, in <module>
    results = [run_trial(seed) for seed in seeds]
               ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa1ea576.py", line 94, in run_trial
    perm_multiplicity = multiplicity((n - m, m), [[random.randint(-10, 10) for _ in range(n)] for _ in r
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa1ea576.py", line 73, in multiplicity
    perm = young_tableau_to_permutation(yt)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa1ea576.py", line 34, in young_tableau
    perm[yt[i][j]] = i * len(yt[i]) + j
    ~~~~^^^^^^^^^^
IndexError: list assignment index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed before collecting data; cannot evaluate support fraction or counterexamples | next: Fix the IndexError in young_tableau_to_permutation by validating tableau dimensions

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	97566
2	preregistration	ollama_remote	qwen3:8b	0	0	24283
3	novelty	ollama_remote	qwen3:8b	0	0	20967
4	novelty	ollama_remote	qwen3:8b	0	0	14928
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17187
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15116
7	judge	ollama_remote	qwen3:8b	0	0	14643

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 204690 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/2746a7cbee6d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2746a7cbee6d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2746a7cbee6d.tar.gz` (if generated)